Meeting 5. Parameter Search and the Gradient

Act 1 Random and Grid Search · Act 2 Solving for Target Scores · Act 3 The Loss Surface and Gradient

About these notes. These instructor-written notes are the course text for CS 301 and are posted on Canvas after each meeting. They include examples and questions discussed in class. The exams cover these notes and your assignment notebook. See the syllabus for the schedule, grading, and other course information.

The loss from meeting 4 lets us compare settings of a unit: a lower loss means its outputs fit the observed labels better under that measure. To find a better setting, we can try parameters at random, solve equations for chosen target scores, or use the slope of the loss to choose a direction. Each approach uses different information about the problem. The gradient collects the slopes with respect to every parameter, even when there are too many parameters to visualize together.

Where the temperature went. A question from Review Quiz 4: why does T not appear in the loss formula — is it absorbed by the score? For $T > 0$, dividing the score by T is the same as dividing all three of the unit's numbers by T:

$$\sigma\left(\frac{\mathbf{w}\cdot\mathbf{x}+b}{T}\right) \;=\; \sigma\left(\frac{\mathbf{w}}{T}\cdot\mathbf{x}+\frac{b}{T}\right).$$

A unit with temperature T is the same function as a unit with temperature 1, weights $\mathbf{w}/T$, and bias b/T. Temperature therefore adds no new functions when the weights and bias are free to vary. It lets us adjust the confidence of a hand-drawn line while keeping its decision boundary fixed.

We use $T = 1$ throughout these notes, with the scale included in the weights and bias. The hand-drawn line high + low = 120 at $T = 10$ is the setting $\mathbf{w} = (0.1, 0.1)$, $b = -12$ at $T = 1$. Its loss is 0.245 and it places 89.7 percent of the days correctly, the numbers meeting 4 quoted, because it is the same unit.

Choosing weights for AND. Review Quiz 2 asked how students found their weights for AND. Seventy people answered, and more than one answer was allowed:

I reasoned from the truth table before drawing anything	35
I tried numbers until the table came out right	25
I looked at which corner was wrong and moved the line toward it	18
I started from a setting I already knew and checked it	16

Trying numbers suggests random search; reasoning from the table gives conditions on the parameters. Moving the line toward a misclassified corner uses information about where the current setting goes wrong.

Act 1. Random and Grid Search

1.1 Parameters and loss

A unit's weights and bias are its **parameters**. A two-input unit has three of them. For any setting of the parameters and a nonempty labeled dataset, we can compute the loss L. On the AND table, scaling a setting whose scores are strictly on the correct side of zero drives the loss toward zero. On the weather data, the best loss measured with the four weather features is about 0.21.

1.2 Random search

Twenty-five people tried numbers until the table came out right. We can describe that approach as a procedure: pick (w_1, w_2, θ) at random inside a box, compute the loss, keep the setting if its loss is the best so far, and repeat. This is **random search**. Its only ingredient beyond meeting 4 is the box. We use $[-6, 6]$ for each of the three numbers, the range the meeting-2 interactive page used.

On the AND table, one run gives the following best-so-far losses. After 1 guess, 1.20. After 10, 0.43. After 100, 0.27. After 1,000, 0.11, and at that point the best setting reproduces the table on all four rows. After 10,000, 0.10.

About 2 percent of this box consists of settings that reproduce AND, so a random guess has roughly one chance in fifty. Across fifty runs, the first guess that reproduces AND took about 52 trials on average, with a range from 1 to 197.

The loss decides which guess to keep, including when both guesses classify the same number of rows correctly.

1.3 Grid search

The systematic version tries every combination of a fixed set of values per parameter: 25 values, $-6, -5.5, \ldots, 6$, for each of the three numbers. This is **grid search**, and it costs $25^3 = 15{,}625$ evaluations of the loss.

The best setting on the grid is $\mathbf{w} = (4, 4)$, $\theta = 6$, with loss 0.096. This is setting B of the meeting-4 sheet with every parameter doubled, also shown in its scaling table at $k = 4$.

1.4 What each extra parameter costs

Grid search has a cost that can be computed before running it. For a rough time estimate, suppose we can evaluate 2,000 settings per second, about the rate measured for a two-input unit on the 6,865 weather days. Holding that rate fixed and trying 25 values per parameter gives:

unit	parameters	evaluations	time
on (high, low)	3	$25^3 \approx 16{,}000$	8 seconds
on all four measurements	5	$25^5 \approx 10^7$	80 minutes
on all six measurements	7	$25^7 \approx 6 \times 10^9$	35 days
a network of three units	9	$25^9 \approx 4 \times 10^{12}$	60 years
forty inputs	41	$25^{41} \approx 10^{57}$	10^{46} years

Each extra parameter multiplies the number of evaluations by 25. The first four rows are two parameters apart, so each costs 625 times the one above it. These times are extrapolations: evaluating a larger unit also takes more work. Even at the assumed rate, nine parameters take about sixty years. A grid finds the best setting among its points; the best setting overall may lie between them or outside the box.

A coarser grid reduces the cost, but 10 values for each of 41 parameters still require 10^{41} evaluations. A smaller box with the same number of grid points improves resolution without reducing the number of evaluations, and it may exclude good settings.

Random search avoids filling a grid. Its success depends on how much of the chosen box contains useful settings. If that fraction is $p > 0$, independent uniform guesses take $1/p$ trials on average to hit one. The fraction depends on the data and the box; the number of parameters alone does not determine it.

1.5 The same searches on the weather data

Random search over (w_1, w_2, b), in the box $w_1, w_2 \in [-0.5, 0.5]$ and $b \in [-50, 50]$, best setting so far:

guesses	best loss	days right
10	2.08	54%
100	0.257	90.5%
1,000	0.240	90.0%
10,000	0.219	91.0%
grid, 15,625	0.220	90.8%

the hand-drawn line	0.245
the best line on (high, low)	0.217

Ten thousand random settings reach a loss of 0.219, against the hand-drawn line's 0.245 and the best line's 0.217, at the price of ten thousand passes over the data. After ten guesses the best setting is right on 54 percent of the days, which is close to a coin flip. The weights multiply temperatures in the tens of degrees, so choices of weights and bias in this box can give large scores with the wrong sign and a high loss. The range and scale of the parameters affect which settings random search is likely to try.

1.6 How search methods choose candidates

Random search keeps the best setting found, but each new guess comes from the same distribution regardless of the losses already observed. Grid search also follows a schedule fixed in advance. Neither uses the current setting's errors to choose a direction of movement, as students did when they moved the line toward a misclassified corner.

Act 2. Solving for Target Scores

2.1 Reasoning from the table

Thirty-five people reasoned from the truth table before drawing anything. In various words, they wrote conditions like these:

x_1	x_2	AND	so
0	0	0	$0 < \theta$
0	1	0	$w_2 < \theta$
1	0	0	$w_1 < \theta$
1	1	1	$w_1 + w_2 \geq \theta$

These four conditions restrict the three parameters. For example, $(w_1, w_2, \theta) = (2, 2, 3)$ satisfies all four and therefore reproduces AND.

2.2 Target scores make linear equations

The conditions above are inequalities: each says which side of zero a score must be on. We can impose a stronger requirement by choosing a specific target score for each row — say -3 on $(0, 0)$, -1 on $(0, 1)$ and $(1, 0)$, and $+1$ on $(1, 1)$. Each row is then one equation in the three unknowns:

$$-\theta = -3, \qquad w_2 - \theta = -1, \qquad w_1 - \theta = -1, \qquad w_1 + w_2 - \theta = 1.$$

The unknowns appear multiplied by numbers and added, and nothing else is done to them. Equations of this form are **linear**; elimination can determine whether they have a solution and find one if they do.

Example 2.1 Weights from target scores

Solve the first three equations, then check the fourth.

The first equation gives $\theta = 3$ directly. The second becomes $w_2 - 3 = -1$, so $w_2 = 2$. The third becomes $w_1 - 3 = -1$, so $w_1 = 2$. Now put these into the fourth: $w_1 + w_2 - \theta = 2 + 2 - 3 = 1$, which is the target. All four equations hold. The setting is $(w_1, w_2, \theta) = (2, 2, 3)$, which is setting B of the meeting-4 sheet; its scores there were $-3, -1, -1, 1$, and that is where these targets came from.

Now change the last target to $+3$ and leave the other three. The first three equations have not changed, so they give $(2, 2, 3)$ again. The fourth now reads $2 + 2 - 3 = 3$, that is, $1 = 3$, which is false. There is no setting that hits all four targets.

Four equations in three unknowns can be inconsistent, as this example shows. To compare settings that miss the chosen scores, we need a measure of their deviations from those targets. Our cross-entropy loss answers a different question: how well the predicted probabilities fit the observed labels.

2.3 What a linear system is

A **linear system** is a set of unknowns $w_1, \ldots, w_n$ together with equations, each of which sets a weighted sum of the unknowns equal to a number. In matrix form the system is $A\mathbf{w} = \mathbf{t}$: each row of A holds the coefficients of one equation, and $\mathbf{t}$ holds the targets.

When there are n equations in n unknowns and the equations are independent — none is a combination of the others — the system has exactly one solution, and a computer finds it by elimination, the method of MATH 337. In NumPy that is a single call:

```
w = np.linalg.solve(A, t)
```

For a dense n-by-n system, elimination takes on the order of n^3 arithmetic operations. It uses the equations' structure instead of enumerating candidate settings. The numerical answer is subject to floating-point rounding; its accuracy also depends on the system.

2.4 Target scores and loss minimization

Labels do not specify target scores. We chose $(-3, -1, -1, 1)$. A truth table gives labels, and a label says which side of zero the score should be on, not what number it should be. Meeting 4 showed that on a truth table the loss prefers scores as far from zero as possible, on the correct side, and "as far as possible" is not a target.

Some weather days have identical inputs and opposite labels. The 6,865 days would give 6,865 score equations in three unknowns. A unit using only high and low gives the same score to days with the same measurements, even when their labels differ. It cannot assign those days target scores on opposite sides of zero. Minimizing loss lets us compare predictions despite these conflicting labels.

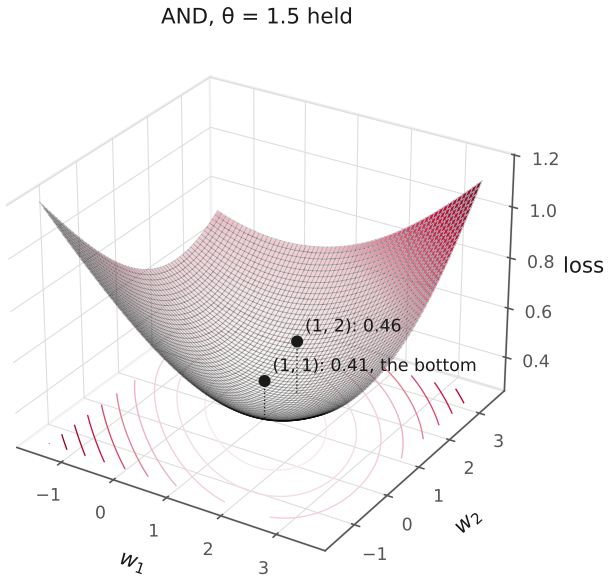
Minimizing cross-entropy gives nonlinear equations. The condition for a finite minimum sets the loss derivatives to zero. Those equations involve the sigmoid of the score and are nonlinear in the parameters, so linear elimination does not solve them.

If you have met least squares. Measuring closeness by the sum of squared differences between the scores and the targets does lead to a linear system, the normal equations. This fits raw scores to chosen targets. Squared error between *sigmoid probabilities* and labels is a different objective and does not give those same linear equations. For probabilities, squared error is bounded by 1 per row; cross-entropy grows without bound as the probability assigned to the observed label approaches zero. We use cross-entropy to give that stronger penalty to confident errors.

Act 3. The Loss Surface and Gradient

3.1 The loss as a surface

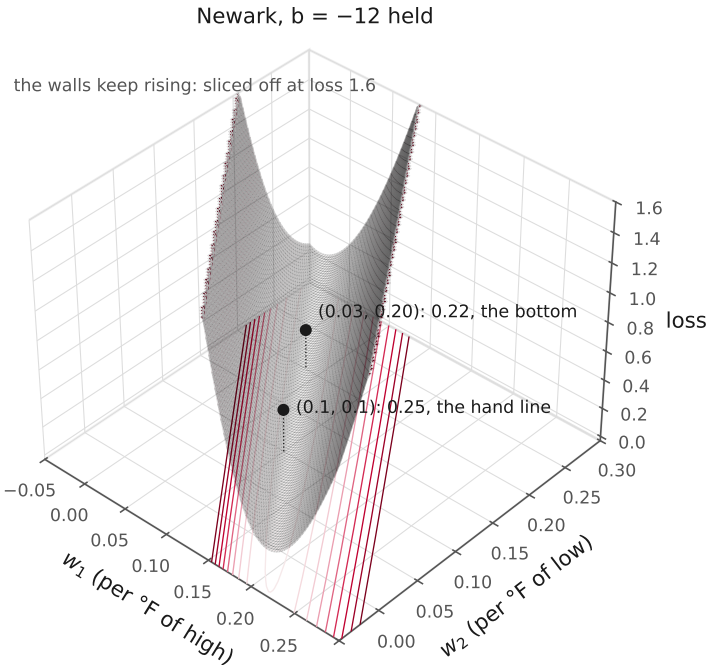
A two-input unit has three parameters. Hold the threshold fixed and two remain, so the loss is a number for every point of the (w_1, w_2) plane. A number over a plane is a surface, and it can be drawn.



This is the loss on the AND table with $\theta = 1.5$ held. It is a bowl. Its lowest point is $\mathbf{w} = (1, 1)$, at loss 0.406 — setting A of the meeting-4 sheet, rounded there to 0.41. Meeting 2's opening unit, $\mathbf{w} = (1, 2)$, sits on the side of the bowl at 0.463.

For AND, scaling all three parameters of setting A together drives the loss toward zero. The surface above has a lowest point at 0.406 because θ is held at 1.5: with the threshold fixed, the weights can grow only so far before the rows $(0, 1)$ and $(1, 0)$ start to fire, and the loss rises again. This two-dimensional slice has a minimum. When all three parameters are free, the loss can approach zero but never reaches it at finite parameter values.

3.2 The same surface on the weather data



The loss on the weather data, over the weights for the high and the low, with $b = -12$ held. The shape is different:

a narrow valley running diagonally, with steep sides. The lowest point is near $\mathbf{w} = (0.03, 0.20)$, at loss 0.218, where the unit places 91.0 percent of the days correctly. The hand-drawn line, $\mathbf{w} = (0.1, 0.1)$, lies in the valley at 0.245. A setting like $(0.3, 0.3)$ is far up the side at loss 6.8, which is why the picture cuts the sides off.

A day's high and low tend to rise and fall together. Increasing one weight while decreasing the other can therefore leave many scores similar, producing a valley of settings with similar losses. Near its floor, the loss changes more slowly along the valley than across it.

You can turn both surfaces with the mouse on the interactive page, *The loss landscape*, linked from Canvas; the floor of the page shows the same loss as a map.

3.3 Slope, and the gradient

Stand at a point of the surface and move a small distance in the w_1 direction only, holding w_2 fixed. The loss changes by some amount. The ratio of the change in the loss to the distance moved, in the limit of small moves, is the slope of the surface in that direction. It is the derivative of L with respect to w_1 with w_2 held fixed, written $\partial L/\partial w_1$ and called a **partial derivative**. It is the derivative of MATH 111, applied to one variable while the other is treated as a constant. The slope in the w_2 direction, $\partial L/\partial w_2$, is defined the same way with the roles exchanged.

The vector of all the slopes,

$$\nabla L \;=\; \left(\frac{\partial L}{\partial w_1},\; \frac{\partial L}{\partial w_2} \right),$$

is the **gradient** of L at that point. When it is nonzero, it points in the direction of steepest local increase, and $-\nabla L$ points in the direction of steepest local decrease. A sufficiently small move in that downhill direction lowers the loss.

The count of misclassified days behaves differently. It stays constant until a day crosses the decision boundary, so its derivative is zero wherever it exists. That slope cannot guide small improvements. The smooth loss supplies local slopes, although a slope at one setting does not tell us the loss at distant settings.

3.4 The slope of one row's contribution

The loss is the mean of the contributions of the rows, so its slope is the mean of their slopes. Take one row and write ℓ for its **contribution to the loss**, as a function of that row's score s. From meeting 4, with $\hat{y} = \sigma(s)$ for the unit's output on the row:

$$\ell(s) \;=\; \begin{cases} -\ln \sigma(s) & \text{if } y = 1, \\ -\ln\big(1 - \sigma(s)\big) & \text{if } y = 0. \end{cases}$$

To differentiate ℓ, first write $\sigma(s) = (1 + e^{-s})^{-1}$ and differentiate:

$$\sigma'(s) \;=\; \frac{e^{-s}}{(1+e^{-s})^2} \;=\; \frac{1}{1+e^{-s}} \cdot \frac{e^{-s}}{1+e^{-s}} \;=\; \sigma(s)\,\big(1 - \sigma(s)\big),$$

since $e^{-s}/(1+e^{-s}) = 1 - 1/(1+e^{-s})$.

Now differentiate each case of ℓ with the chain rule. For a row with label 1,

$$\frac{d\ell}{ds} \;=\; -\frac{\sigma'(s)}{\sigma(s)} \;=\; -\big(1 - \sigma(s)\big) \;=\; \hat{y} - 1.$$

For a row with label 0,

$$\frac{d\ell}{ds} \;=\; \frac{\sigma'(s)}{1-\sigma(s)} \;=\; \sigma(s) \;=\; \hat{y} - 0.$$

In both cases,

$$\frac{d\ell}{ds} \;=\; \hat{y} - y.$$

Define the row's **error** as its output minus its label, $\hat{y} - y$. The slope of a row's contribution to the loss, with respect to that row's score, is the row's error.

3.5 How large the error can be

A row the unit already gets right with confidence has $\hat{y}$ close to y, so both its error and $d\ell/ds$ are close to 0.

A row the unit gets wrong with confidence has $\hat{y}$ close to $1 - y$, so its error is close to $+1$ if the label is 0 and close to -1 if the label is 1. Its derivative with respect to the score has magnitude close to 1.

The output $\hat{y}$ is strictly between 0 and 1, so the error satisfies $|\hat{y} - y| < 1$ on every row. The bound is approached but never reached for a finite score in exact arithmetic. This bounds $d\ell/ds$, the derivative with respect to the *score*. A row's contribution to a weight derivative also depends on the size of its input, as the chain rule shows below.

3.6 The gradient of the loss

The score is $s = w_1 x_1 + w_2 x_2 + b$, so $\partial s/\partial w_1 = x_1$, $\partial s/\partial w_2 = x_2$, and $\partial s/\partial b = 1$. The loss is the mean of the row contributions, $L = \frac{1}{n}\sum_i \ell_i$. By the chain rule, the slope of L with respect to w_j is the mean over the rows of (slope of ℓ_i with respect to s_i) times (slope of s_i with respect to w_j):

$$\frac{\partial L}{\partial w_j} \;=\; \frac{1}{n}\sum_{i=1}^{n} (\hat{y}_i - y_i)\, x_{ij}, \qquad \frac{\partial L}{\partial b} \;=\; \frac{1}{n}\sum_{i=1}^{n} (\hat{y}_i - y_i).$$

Here x_{ij} is the j-th input of row i. In matrix form, with X the array whose rows are the inputs and $\hat{\mathbf{y}} - \mathbf{y}$ the vector of errors,

$$\nabla_{\mathbf{w}} L \;=\; \frac{1}{n}\, X^{\mathsf{T}} (\hat{\mathbf{y}} - \mathbf{y}),$$

where multiplying by X^{T} sums the error-weighted inputs for each feature. The factor $1/n$ averages these contributions over the rows.

Each row contributes its input vector $\mathbf{x}_i$ multiplied by its error. For label 1, the error is negative, so that row's contribution to the downhill direction points along $+\mathbf{x}_i$, raising its score. For label 0, it points along $-\mathbf{x}_i$, lowering its score. The full direction combines all rows; their contributions can reinforce or oppose one another. Large inputs can give a row substantial influence even when its error is small. Moving a sufficiently small distance against a nonzero gradient lowers the mean loss, without necessarily improving every row.

Example 3.1 At meeting 2's opening unit

Compute the gradient at $\mathbf{w} = (1, 2)$, $\theta = 1.5$, the unit that places the corner $(0, 1)$ on the wrong side. Sigmoid values are rounded to two decimals.

row	label y	score s	output $\hat{y} = \sigma(s)$	error $\hat{y} - y$
$(0, 0)$	0	-1.5	0.18	$+0.18$
$(0, 1)$	0	$+0.5$	0.62	$+0.62$
$(1, 0)$	0	-0.5	0.38	$+0.38$
$(1, 1)$	1	$+1.5$	0.82	-0.18

For $\partial L/\partial w_1$, multiply each error by that row's x_1 and average: only the rows $(1, 0)$ and $(1, 1)$ have $x_1 = 1$, so

$$\frac{\partial L}{\partial w_1} = \frac{0.38 - 0.18}{4} = 0.05.$$

For $\partial L/\partial w_2$, the rows with $x_2 = 1$ are $(0, 1)$ and $(1, 1)$:

$$\frac{\partial L}{\partial w_2} = \frac{0.62 - 0.18}{4} = 0.11.$$

For the bias, every row has $x = 1$ in the bias position, so $\partial L/\partial b$ is the mean of all four errors, $(0.18 + 0.62 + 0.38 - 0.18)/4 = 0.25$. Since $\theta = -b$, the slope with respect to θ is the negative of this: $\partial L/\partial \theta = -0.25$.

Downhill is $-\nabla L$: decrease w_2 (the largest slope among the weights), decrease w_1 a little, and increase θ. On the unit square of meeting 2, decreasing w_2 and raising θ move the line away from the origin on the x_2 side — toward the corner $(0, 1)$, the one that was wrong. Moving the line toward the wrong corner resembles the adjustment eighteen students described at meeting 2. The gradient computes a specific direction using all four rows' errors, and the same calculation works with more inputs.

Example 3.2 The slope, two ways

At $\mathbf{w} = (1, 2)$, $\theta = 1.5$, the loss is about 0.46. We can compare the formula for $\partial L/\partial w_2$ with a finite difference estimate at this setting.

As a finite difference. Move w_2 up to 2.5 and down to 1.5, leaving $w_1 = 1$ and $\theta = 1.5$, and compute the loss at each, using sigmoid and logarithm values rounded to two decimals.

At $w_2 = 2.5$ the scores are $-1.5, +1, -0.5, +2$. The probabilities of the observed labels are $0.82, 0.27, 0.62, 0.88$ (on the second row the label is 0 and $\sigma(1) = 0.73$, so $p = 1 - 0.73 = 0.27$). The contributions $-\ln p$ are $0.20, 1.31, 0.48, 0.13$, and the loss is their mean, $2.12/4 = 0.53$.

At $w_2 = 1.5$ the scores are $-1.5, 0, -0.5, +1$. The probabilities are $0.82, 0.50, 0.62, 0.73$; the contributions are $0.20, 0.69, 0.48, 0.32$; the loss is $1.69/4 = 0.42$.

The slope is approximately the change in the loss over the change in w_2:

$$\frac{\partial L}{\partial w_2} \approx \frac{0.53 - 0.42}{2.5 - 1.5} = 0.11.$$

By the formula. At $w_2 = 2$ the outputs are $0.18, 0.62, 0.38, 0.82$, as in Example 3.1, and the rows with $x_2 = 1$ are the second and the fourth: $(0.62 - 0.18)/4 = 0.11$.

The two agree to two decimals. The finite difference uses a step of 0.5 in each direction, for a total interval of 1. Using unrounded loss values gives an estimate of about 0.1084; evaluating the derivative formula gives about 0.1100. One-sided estimates from those unrounded values are about 0.1323 upward and 0.0845 downward, so the two-sided estimate is more accurate here.

A two-sided finite difference needs two loss evaluations per parameter. Forty parameters require eighty evaluations for a full gradient estimate. A one-sided estimate needs forty additional evaluations after the loss at the starting point is known. The formula computes all forty derivatives in one pass over the data, reusing each row's error across the inputs. Finite differences provide a useful check on its implementation.

3.7 Why linear elimination does not find the minimum

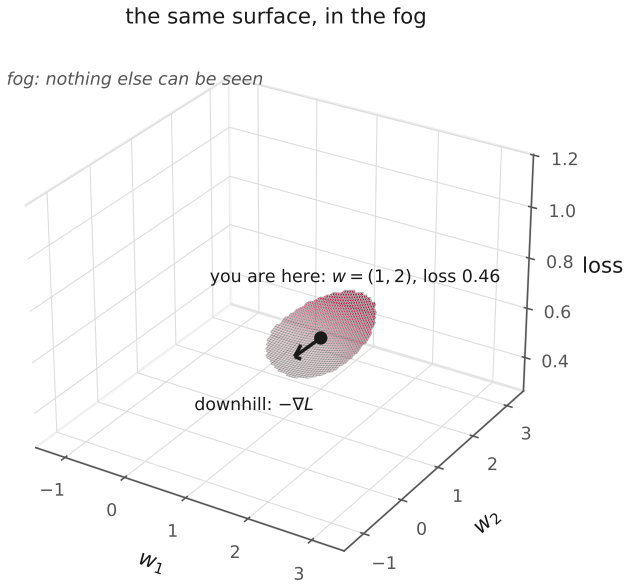
At a finite minimum of this differentiable loss, with all parameters free to vary, the gradient must be zero. The weight equations are

$$\sum_{i=1}^{n} (\hat{y}_i - y_i)\, x_{ij} \;=\; 0 \qquad \text{for every } j.$$

The bias equation is $\sum_i (\hat{y}_i - y_i) = 0$. There is one equation per parameter, but $\hat{y}_i = \sigma(w_1 x_{i1} + w_2 x_{i2} + b)$, so the unknowns occur inside the sigmoid. These equations are nonlinear and cannot be passed to `np.linalg.solve`. This limits the linear-system approach; it does not rule out numerical methods for nonlinear equations or special cases with explicit solutions.

We can still compute the gradient at any finite setting without first finding a minimum. As the AND example shows, a finite minimum need not exist when all parameters are free.

3.8 Following the local gradient



The figure shows a small neighborhood of $(1, 2)$ on the AND loss surface, with an arrow in the direction $-\nabla L$. An iterative method can take a short step in that direction and recompute the gradient at the new parameters. Each step uses local information about the loss.

A unit with forty inputs has forty-one parameters. We can draw slices of its loss landscape, but cannot display all forty-one parameter directions together as an ordinary surface. The gradient calculation still averages each row's error times its inputs, with a separate component for the bias.

On the interactive page, switch **fog** on. The surface disappears and a patch of ground around the marked point remains, with the downhill arrow on it. Press **step downhill**: the page moves the point against the gradient and computes the gradient again at the new point. The distance depends on the gradient's size and the page's learning-rate setting. At the default learning rate, from $(1, 2)$ on the AND surface the loss goes 0.463, 0.438, 0.424, 0.417, 0.413, and the point moves toward $(1, 1)$ without the surface ever being shown. Switch the fog off afterwards to see the path on the full surface. On the weather surface, from the hand-drawn line, the loss goes 0.245, 0.231, 0.229, 0.229, and then decreases slowly along the valley toward 0.218. Press **random guess** to sample new settings from the search box. Each draw is independent of the preceding setting and its loss.

3.9 Comparing the methods

Grid search checks every setting on a chosen grid, at a cost that grows exponentially with the number of parameters. Random search samples a chosen box and keeps its best result; its chance of finding a useful setting depends on the fraction of the box those settings occupy. Linear elimination solves consistent equations for chosen target scores, but the equations for minimizing cross-entropy are nonlinear.

The gradient gives a local direction from the errors at the current setting. Computing it does not require knowing where the minimum is or drawing the loss landscape. The interactive examples take steps using this direction; a complete search procedure also needs a choice of step size and a stopping condition.

Act 4. Practice

4.1 Search and derivative methods

Matching. Match each method to its description.

Methods: random search; grid search; finite differences; analytic gradient.

Descriptions:

A. Approximate a derivative by evaluating the loss at nearby parameter values.

B. Evaluate every combination from specified lists of parameter values.

C. Compute derivatives using formulas derived from the model and loss.

D. Evaluate sampled parameter vectors.

4.2 Target scores and classification

True or false, with explanation. We choose numerical target scores for the training examples and obtain an inconsistent system of linear equations. Therefore, no threshold neuron can classify all the examples correctly.

4.3 A zero-valued feature

Application. One example has a large prediction error, but its j-th feature is zero. Using

$$\frac{\partial \ell}{\partial w_j} = (q - y)x_j,$$

explain why this example makes no contribution to the derivative with respect to w_j. Can it still contribute to the bias derivative?

4.4 A zero full-batch derivative

Short explanation. The full-batch derivative with respect to one weight is zero. Must every training example's contribution to that derivative also be zero? Explain a situation in which the contributions are nonzero but their average is zero.